

MECHATRONICS AND IOT

ASSIGNMENT - 14

WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM

1. Project Overview

The Water Level Detection and Flood Alert System is an IoT-based safety system designed to monitor the water level in tanks, rivers, reservoirs, dams, and other water bodies. The system uses an HC-SR04 ultrasonic sensor to measure the distance between the sensor and the water surface.

An ESP8266 NodeMCU acts as the main controller. It processes the ultrasonic sensor readings and determines the current water-level condition. LEDs and a buzzer provide local visual and audible alerts when the water level reaches critical conditions.

The system can also connect to Adafruit IO Cloud through Wi-Fi, allowing the water level and alert status to be monitored remotely.

Main components

- ESP8266 NodeMCU 1
- HC-SR04 Ultrasonic Sensor 1
- Buzzer 1
- LED Indicators 2
- OLED DisplayGSM Module 1
- Breadboard 1
- Jumper Wires

2. Project Statement

Flooding and water overflow can cause damage to agricultural land, buildings, roads, electrical infrastructure, and other property. Conventional water-level monitoring often depends on manual observation, which may not provide sufficient warning during rapidly rising water levels.

Therefore, an automated system is required that can:

- Continuously monitor water levels.
- Detect abnormal increases in water level.
- Provide early warning before overflow.
- Generate audible and visual alerts.
- Send water-level information to the cloud.
- Allow remote monitoring.
- Reduce the need for continuous manual observation.

The proposed system addresses these requirements using an ultrasonic sensor, ESP8266, alert indicators, and IoT connectivity.

3. Proposed Solution

The proposed Water Level Detection and Flood Alert System is an IoT-based monitoring system designed to continuously measure water levels and provide early warnings during critical or overflow conditions.

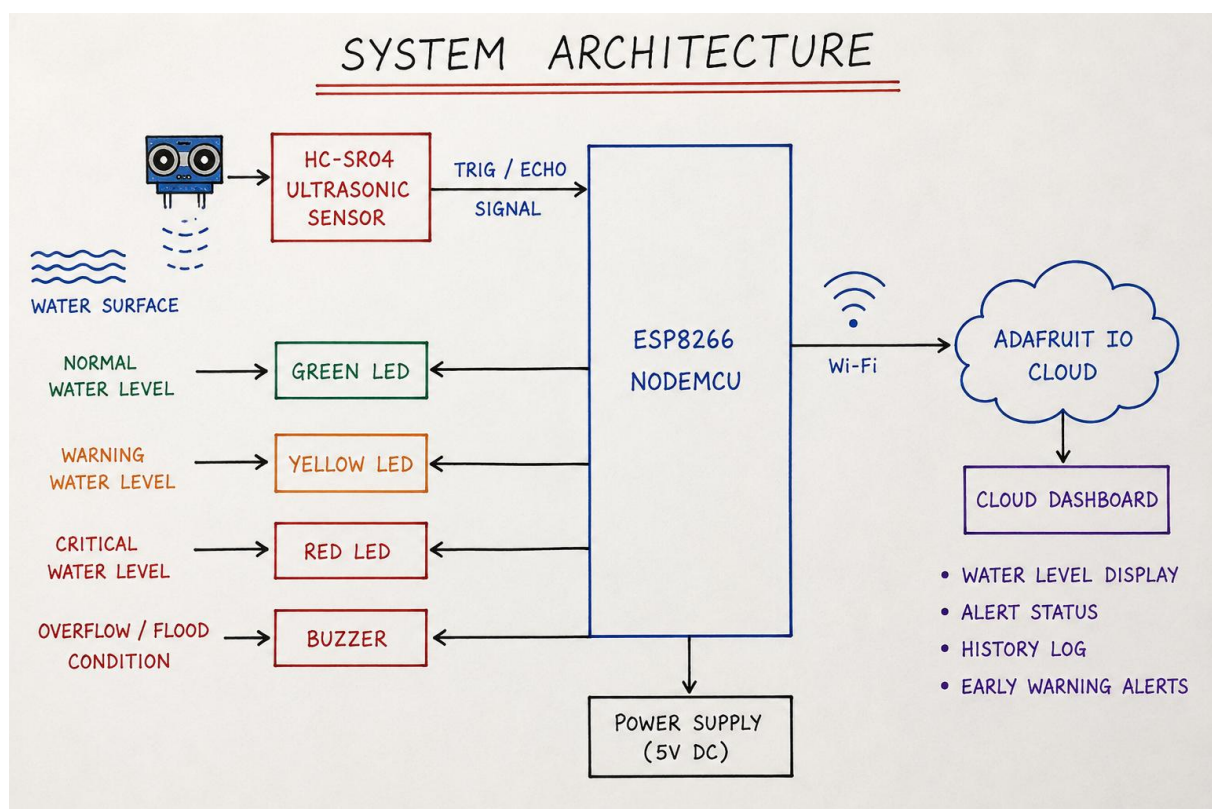
An HC-SR04 ultrasonic sensor is installed above the water surface. It transmits ultrasonic waves toward the water and measures the time taken for the reflected waves to return. Based on this time, the distance between the sensor and the water surface is calculated. As the water level rises, the measured distance decreases.

An ESP8266 NodeMCU acts as the main controller. It processes the sensor readings and compares the calculated water level with predefined threshold values.

The system provides different warning levels:

- **Normal Water Level:** Green LED is activated.
- **Warning Level:** Yellow LED is activated to indicate rising water.
- **Critical Level:** Red LED and buzzer are activated.
- **Overflow/Flood Condition:** A high-level alert is generated and the information is transmitted to the Adafruit IO Cloud through Wi-Fi.

4. System Architecture:



5. Circuit Table:

CIRCUIT TABLE

1. ULTRASONIC SENSOR CONNECTION

Component	Pin	ESP8266 NodeMCU Pin	Function
HC-SR04 Ultrasonic Sensor	VCC	5V	Power Supply (5V)
	GND	GND	Ground
	TRIG	D5	Trigger Signal
	ECHO	D6 (via Voltage Divider)	Echo Signal

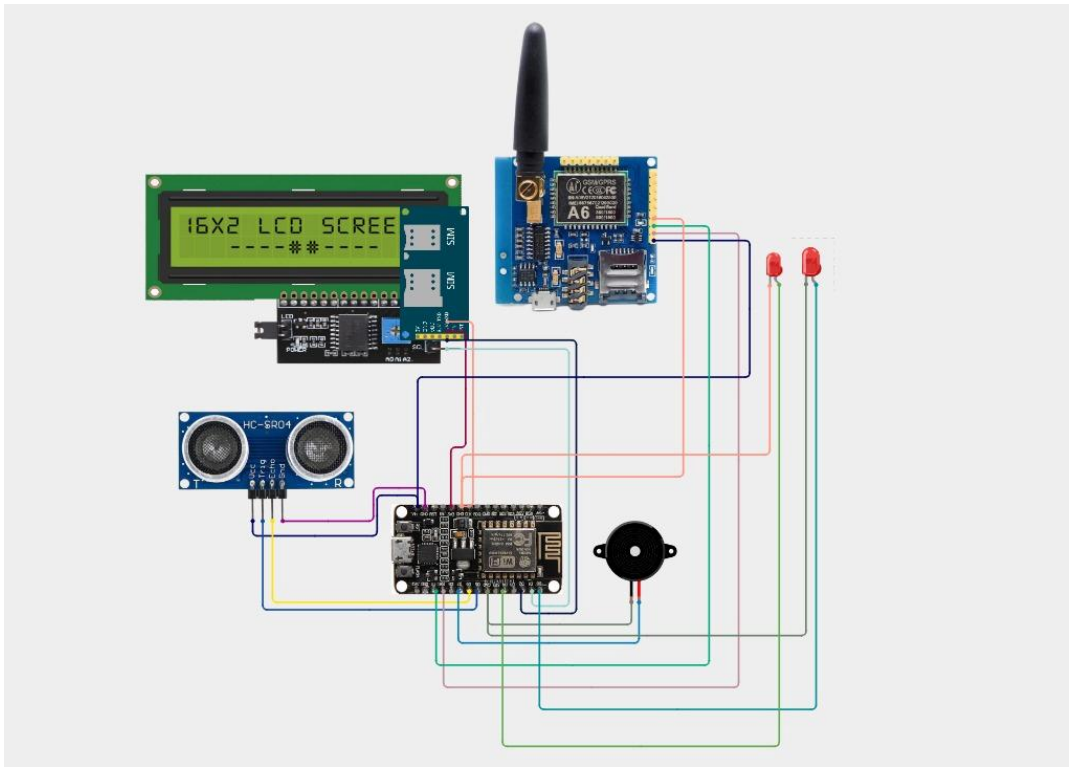
2. LED & BUZZER CONNECTION

Component	Pin	ESP8266 NodeMCU Pin	Function
Green LED	Anode (+)	D1 (Through 220Ω)	Indicates Safe Level
	Cathode (-)	GND	Ground
Red LED	Anode (+)	D2 (Through 220Ω)	Indicates Flood / High Level
	Cathode (-)	GND	Ground
Buzzer	Positive (+)	D7	Flood Alert
	Negative (-)	GND	Ground

3. CLOUD & POWER CONNECTION

Component	Pin	ESP8266 NodeMCU Pin	Function
Wi-Fi	Built-in	ESP8266	Internet Connection
Adafruit IO	Internet	ESP8266	Cloud Monitoring
Power Supply	5V DC	VIN / 5V	System Power

6. Circuit Design:



7. Algorithm:

- Start the system.
- Initialize the ESP8266.
- Initialize the ultrasonic sensor.
- Initialize LEDs and buzzer.
- Connect ESP8266 to Wi-Fi.
- Connect to Adafruit IO.
- Send a trigger pulse from the HC-SR04.
- Measure the echo duration.
- Calculate the distance to the water surface.
- Calculate the water level.
- Compare the water level with predefined thresholds.
- If the level is normal, turn ON the green LED.
- If the level reaches the warning threshold, turn ON the yellow LED.
- If the level reaches the critical threshold, turn ON the red LED and buzzer.
- Upload the water-level value and status to Adafruit IO.
- Repeat the process continuously.

8. Coding:

```
#include <ESP8266WiFi.h>

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"

// =====

// WIFI
```

```
// =====  
#define WIFI_SSID    "OnePlus Nord CE5 7C5C"  
#define WIFI_PASSWORD "qkts5737"  
// =====  
// ADAFRUIT IO  
// =====  
#define AIO_SERVER    "io.adafruit.com"  
#define AIO_SERVERPORT 1883  
#define AIO_USERNAME  "Sivasaravanan"  
#define AIO_KEY       "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"  
// =====  
// PINS  
// =====  
#define MQ135_PIN A0  
#define GREEN_LED D5  
#define RED_LED D6  
// =====  
// MQTT  
// =====  
WiFiClient client;  
Adafruit_MQTT_Client mqtt(  
    &client,  
    AIO_SERVER,  
    AIO_SERVERPORT,  
    AIO_USERNAME,  
    AIO_KEY  
);  
// =====  
// FEEDS
```

```

// =====
Adafruit_MQTT_Publish mq135Feed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/mq135"
  );
Adafruit_MQTT_Publish airQualityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/air-quality"
  );
// =====
// SETUP
// =====
void setup()
{
  Serial.begin(9600);
  delay(1000);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  Serial.println();
  Serial.println("=====");
  Serial.println(" AIR POLLUTION MONITORING");
  Serial.println("=====")
  // -----
  // CONNECT WIFI
  // -----

```

```
Serial.print("Connecting to WiFi");
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}
Serial.println();
Serial.println("WiFi CONNECTED!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
// -----
// SENSOR WARM UP
// -----
Serial.println();
Serial.println("MQ135 sensor warming up...");
delay(10000);
Serial.println("Sensor ready!");
}
// =====
// MQTT CONNECTION
// =====
void MQTT_connect()
{
    if (mqtt.connected())
    {
        return;
    }
    Serial.println();
```

```

Serial.print("Connecting to Adafruit IO...");

int8_t ret;

while ((ret = mqtt.connect()) != 0)
{
    Serial.println();

    Serial.print("MQTT connection failed: ");

    Serial.println(mqtt.connectErrorString(ret));

    mqtt.disconnect();

    delay(5000);

    Serial.print("Retrying...");
}

Serial.println();

Serial.println("Adafruit IO CONNECTED!");
}

// =====
// LOOP
// =====

void loop()
{
    // Connect MQTT
    MQTT_connect();

    // -----

    // READ MQ135
    // -----

    int gasValue = analogRead(MQ135_PIN);

    Serial.println();

    Serial.println("=====");

    Serial.print("MQ135 Value: ");

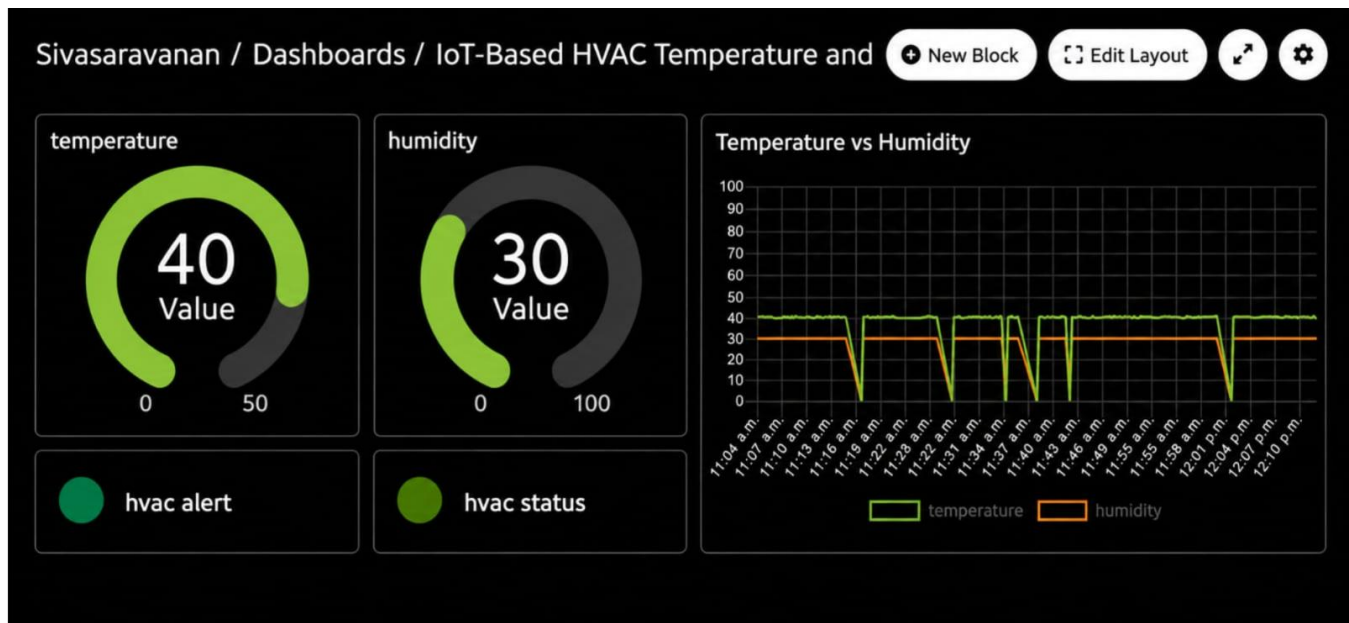
    Serial.println(gasValue);
}

```

```
// -----  
// AIR QUALITY  
// -----  
if (gasValue < 400)  
{  
    digitalWrite(GREEN_LED, HIGH);  
    digitalWrite(RED_LED, LOW);  
    Serial.println("Air Quality: GOOD");  
    // Publish gas value  
    if (mq135Feed.publish(gasValue))  
    {  
        Serial.println("MQ135 data sent successfully!");  
    }  
    else  
    {  
        Serial.println("ERROR: MQ135 data NOT sent!");  
    }  
    // Publish air quality  
    if (airQualityFeed.publish("GOOD"))  
    {  
        Serial.println("Air quality sent successfully!");  
    }  
  
    Else  
{  
    Serial.println("ERROR: Air quality NOT sent!");  
    }  
}  
else
```

```
{  
  digitalWrite(GREEN_LED, LOW);  
  digitalWrite(RED_LED, HIGH);  
  
  Serial.println("Air Quality: POOR");  
  
  // Publish gas value  
  if (mq135Feed.publish(gasValue))  
  {  
    Serial.println("MQ135 data sent successfully!");  
  }  
  else  
  {  
    Serial.println("ERROR: MQ135 data NOT sent!");  
  }  
  
  // Publish air quality  
  if (airQualityFeed.publish("POOR"))  
  {  
    Serial.println("Air quality sent successfully!");  
  }  
  else  
  {  
    Serial.println("ERROR: Air quality NOT sent!");  
  }  
}  
Serial.println("=====");  
delay(5000);  
}
```

9. System Working :



When the system is powered ON, the ESP8266 initializes the ultrasonic sensor, LEDs, buzzer, and Wi-Fi connection.

The HC-SR04 sensor sends an ultrasonic pulse toward the water surface. The pulse reflects from the water surface and returns to the sensor.

The ESP8266 calculates the distance based on the echo time. When the water level is low or normal, the green LED remains ON.

As the water level rises and reaches the warning threshold, the yellow LED is activated to indicate that attention is required.

When the water reaches the critical threshold, the red LED and buzzer are activated. This provides an immediate local warning.

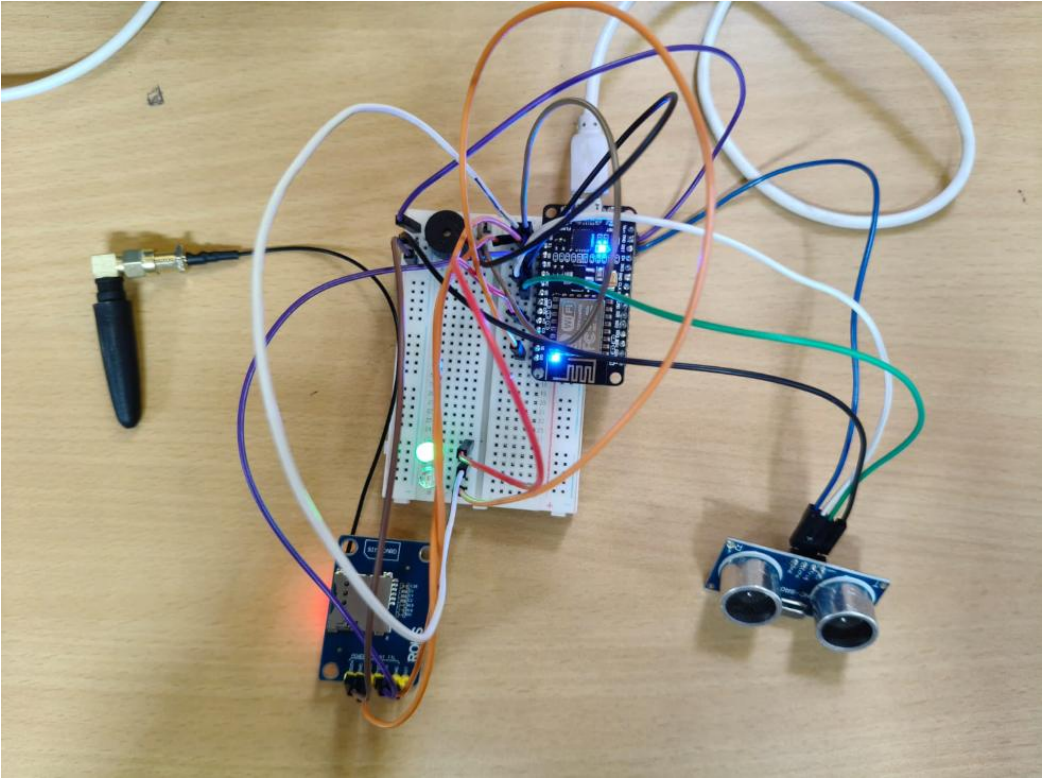
At the same time, the ESP8266 uploads the measured water level and system status to Adafruit IO. The user can view the information remotely through the cloud dashboard. The measurement and alert process continues repeatedly.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component		Specification / Description	Quantity
1.		ESP8266 NodeMCU	ESP8266 Wi-Fi Development Board	1
2.		MQ135 Gas Sensor	Gas Sensing Module (Analog Output)	1
3.		Green LED	5mm LED (Green)	1
4.		Red LED	5mm LED (Red)	1
5.		Resistor	220 Ω Resistor (for LEDs)	2
6.		Breadboard	Solderless Breadboard	1
7.		Jumper Wires	Male to Male Jumper Wires	As required
8.		USB Cable	Micro USB Cable for Power & Programming	1
9.		Wi-Fi Connection	Internet Connection for ESP8266	1
10.		Adafruit IO Cloud	Cloud Platform for Data Monitoring	1

11. Prototype Implementation:



The prototype can be constructed using an ESP8266 NodeMCU, HC-SR04 ultrasonic sensor, three LEDs, buzzer, and supporting components.

The ultrasonic sensor is mounted at a fixed height above a water container representing a tank, reservoir, or water body.

The ESP8266 is connected to the sensor and indicators. The system is powered through a suitable 5 V supply.

Different water levels can be simulated by adding or removing water from the container.

- Low water → Green LED.
- Medium water → Yellow LED.
- High water → Red LED.
- Critical/overflow level → Red LED + buzzer.

The ESP8266 simultaneously sends the measured water-level data to the Adafruit IO dashboard through Wi-Fi.

12. Results and Performance Analysis:

The developed Smart Irrigation and Water Management System successfully demonstrates automatic irrigation based on soil moisture conditions.

During testing, the soil moisture sensor provided different readings for dry and wet soil. When the soil was dry, the ESP8266 detected the condition and activated the relay, causing the water pump to turn ON. After sufficient water was supplied and the soil became moist, the controller switched OFF the relay and stopped the pump.

The automatic switching of the pump helps prevent unnecessary irrigation and reduces water wastage.

The system also provides the possibility of remote monitoring through a cloud platform. Soil moisture readings and pump status can be stored and displayed on a cloud dashboard for observation and analysis.

1. Water-Level Detection: The HC-SR04 continuously measures the distance between the sensor and water surface. This enables real-time monitoring of changing water levels.

2. Early Warning: The warning threshold provides an indication before the water reaches the critical level, allowing preventive action.

3. Automatic Alert: The red LED and buzzer provide immediate local notification during critical water-level conditions.

4. IoT Monitoring: The ESP8266 sends water-level information to Adafruit IO, allowing remote monitoring through a cloud dashboard.

5. Response: The system can repeatedly measure water level and respond automatically when threshold conditions are reached.

6. Cost Effectiveness: The system uses low-cost and easily available components, making it suitable for educational prototypes and small-scale monitoring applications.

Overall Result

The Water Level Detection and Flood Alert System provides an effective method for continuously monitoring water levels and generating early warnings. The combination of ultrasonic sensing, ESP8266 processing, LED/buzzer alerts, and IoT monitoring makes the system suitable for applications such as water tanks, reservoirs, dams, and flood-prone areas.

For a real river or dam deployment, the prototype would need additional environmental protection, reliable power backup, weatherproof sensor mounting, communication redundancy, and properly calibrated site-specific warning thresholds.

